

SRAG

Sanskrit RAG with Graph-Enhanced Linguistic Re-ranking

A Hybrid Retrieval-Augmented Generation System
for the Bhagavad Gita

Version	0.2.0
Pipeline	LangGraph + Adaptive Re-ranking
Embedding Model	sanganaka/bge-m3-sanskritFT
Generation Model	MiMo v2.5
Corpus	3507 chunks across 18 chapters
Tests	48/48 passing
Lint	Ruff clean (0 errors)

Table of Contents

1. Project Overview
2. System Architecture
3. Data Pipeline
4. Retrieval System
5. Adaptive Re-ranking
6. LangGraph Orchestration
7. Answer Generation
8. Evaluation Results
9. Project Structure
10. Configuration
11. API Reference
12. Getting Started

1. Project Overview

SRAG is a hybrid Retrieval-Augmented Generation system designed for the Bhagavad Gita. It combines three complementary retrieval methods (semantic vector search, knowledge graph traversal, and BM25 lexical matching) with a novel 9-feature adaptive re-ranking mechanism and LangGraph orchestration to generate scholarly, well-cited answers.

The system processes queries in any language (English, Hindi, Sanskrit in IAST or Devanagari), converts them to IAST transliteration, extracts philosophical concepts (dharma, karma, yoga, etc.), and generates structured markdown answers with proper verse citations.

Key Innovations:

- **Adaptive Retrieval:** Weights shift dynamically based on query type — vector-heavy for definitional queries, graph-heavy for factual queries, BM25-heavy for complex long queries.
- **9-Feature Re-ranking:** Combines vector score, graph centrality, BM25 score, lemma overlap, morphological match, compound match, commentary relevance, concept overlap, and graph centrality.
- **LangGraph State Machine:** Iterative query expansion when confidence is low (max 2 iterations), enabling self-correction.
- **Separation of Concerns:** Verses and commentaries are treated as separate content types — verses form the primary answer, commentaries appear only as brief scholarly context at the end.
- **Sanskrit-Aware Processing:** IAST-aware tokenization with suffix stripping, morphological feature extraction, and concept-aware query expansion.

2. System Architecture

The SRAG pipeline consists of five major stages: Query Processing, Hybrid Retrieval, Fusion, Re-ranking, and Generation. Each stage is modular and can be configured independently.

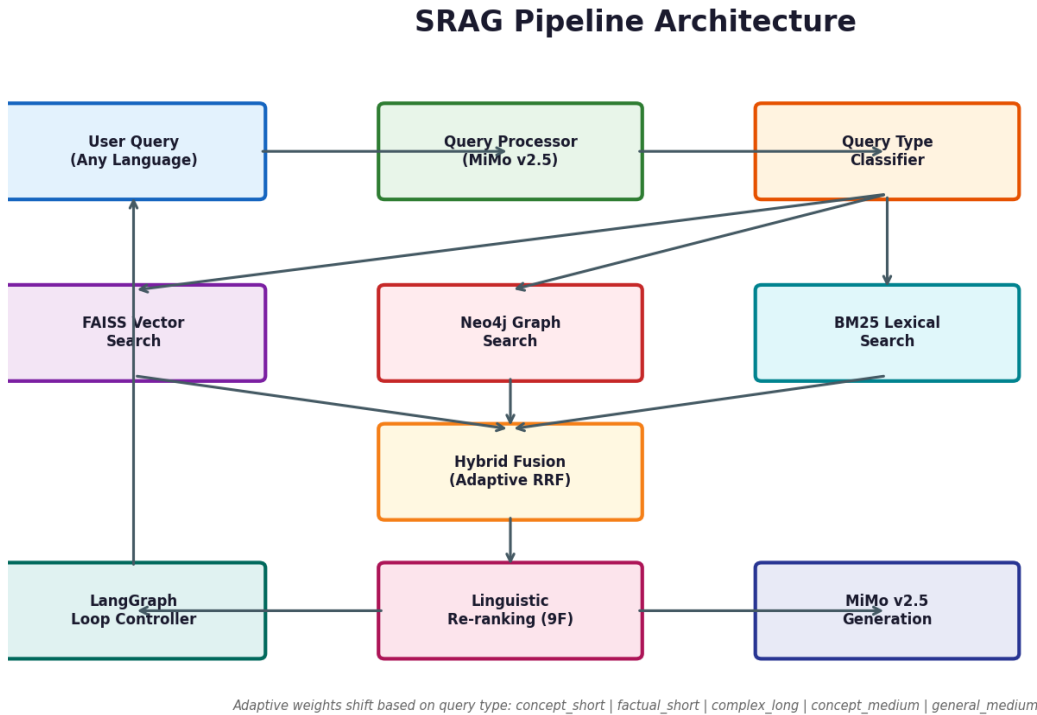


Figure 1: SRAG Pipeline Architecture

2.1 Query Processing

The QueryProcessor uses MiMo v2.5 to analyze user queries in any language. It performs: (1) language detection (English/Hindi/IAST/Devanagari), (2) IAST conversion via LLM, (3) concept extraction (26 Bhagavad Gita philosophical concepts), and (4) query type classification (concept_short, factual_short, complex_long, concept_medium, general_medium).

2.2 Hybrid Retrieval

Three parallel retrieval strategies are executed simultaneously:

- **Vector Search (FAISS):** Encodes the query in Devanagari using bge-m3-sanskritFT, searches the FAISS IndexFlatIP for the 50 most similar chunks by cosine similarity.
- **Graph Search (Neo4j):** Executes full-text search on IAST text, concept neighborhood traversal, and combined queries. Returns up to 50 results with graph scores.
- **BM25 Search:** Tokenizes the IAST query with suffix-aware lemma expansion, searches the BM25Okapi index for 50 matching chunks.

3. Data Pipeline

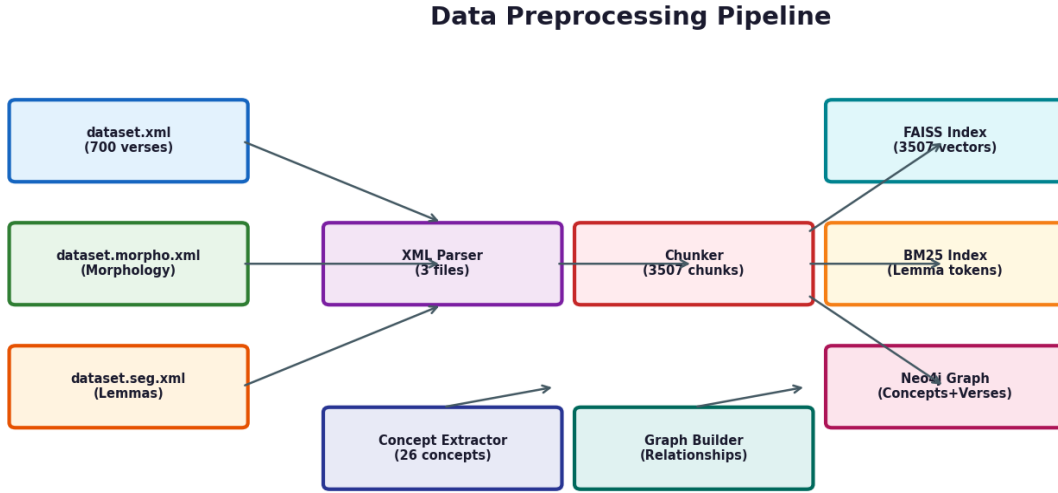


Figure 2: Data Preprocessing Pipeline

3.1 Source Corpus

The Bhagavad Gita corpus consists of three XML files: (1) dataset.xml containing 700 verses in IAST and Devanagari with commentaries from three scholars (Sridhara Swamin — Advaita tradition, Visvanatha Chakravarti — Acintya-bhedabheda, Baladeva Vidyabhushana — Acintya-bhedabheda), (2) dataset.lemma-morphosyntax.xml with morphological annotations (case, gender, tense, mood) for each token, and (3) dataset.segmentation-lemma.xml with lemma-segmented text.

3.2 Chunking Strategy

The XML parser extracts verses, commentaries, and morphological data. The chunker creates 3507 chunks, each containing: IAST text, Devanagari text, lemmas (for BM25), morphological profile, concept associations, and chunk type (verse/commentary/combined). Verse chunks are prioritized over commentary chunks for better definitions.

3.3 Index Building

FAISS Index: Uses `sanganaka/bge-m3-sanskritFT` (1024-dimensional Sanskrit embeddings). Encodes Devanagari text of all chunks. Built as `IndexFlatIP` for cosine similarity. Takes approximately 75 minutes on CPU for 3507 chunks.

BM25 Index: Uses `rank-bm25` library with IAST-aware tokenization. Query expansion strips Sanskrit suffixes (a■, am, ena, ■ya, etc.) and adds root + 'a' forms.

Neo4j Graph: Nodes: Verse, Chapter, Concept, Commentator. Relationships: VERSE_OF_CHAPTER, HAS_CONCEPT, HAS_COMMENTARY, MENTIONS_CONCEPT. Full-text index `verse_text_ft` on `text_jast` field.

4. Retrieval System

The HybridRetriever combines results from all three retrieval methods using Reciprocal Rank Fusion (RRF) with adaptive weights.

4.1 Reciprocal Rank Fusion

RRF combines ranked lists from multiple retrievers into a single ranking. For each document, the RRF score is: $\text{rrf_score} = \sum (\text{weight_i} / (k + \text{rank_i}))$, where $k = 60$ (default). Documents appearing in multiple retrievers get boosted scores.

4.2 Adaptive Weights

Weights shift based on query type. Short definitional queries favor vector similarity (0.50). Factual queries favor graph relationships (0.40). Complex long queries favor BM25 exact matching (0.40).

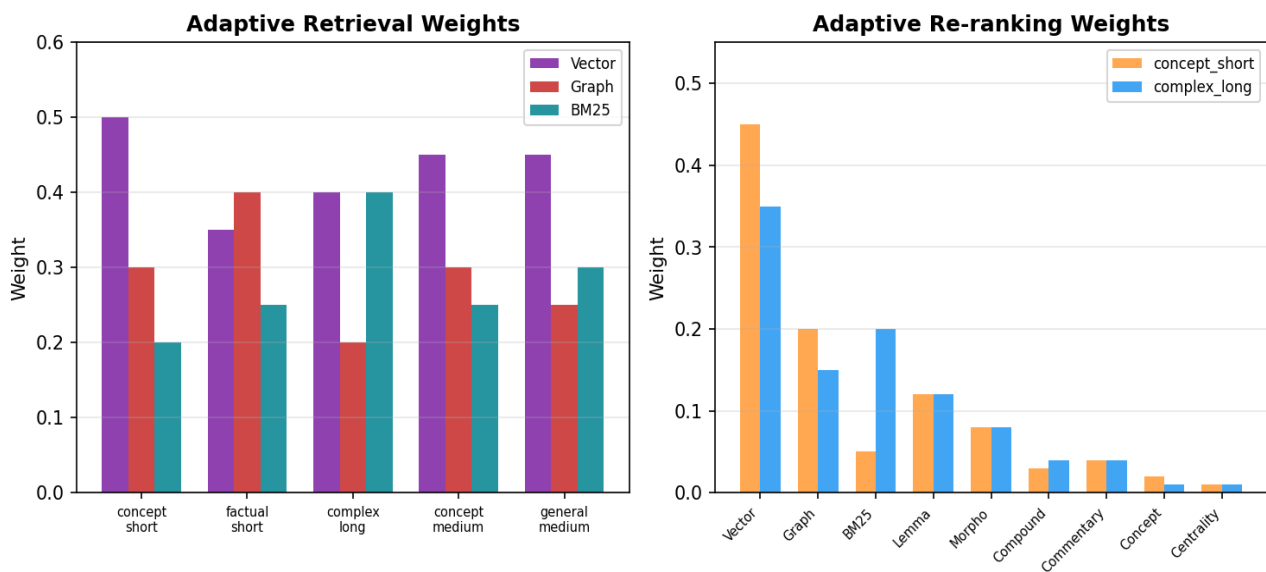


Figure 3: Adaptive Retrieval and Re-ranking Weights

4.3 Score Propagation

Each fused result carries individual scores from all three retrievers: `vector_score`, `graph_score`, `bm25_score`, and the combined `rrf_score`. This ensures downstream components can reason about the source of each result.

5. Adaptive Re-ranking

The LinguisticReranker extracts 9 features per candidate and applies dynamic weights based on query type. This is the core innovation of SRAG.

#	Feature	Weight	Description
1	score_vector	0.40	FAISS cosine similarity
2	score_graph	0.20	Neo4j graph retrieval score
3	score_bm25	0.10	BM25 lexical matching score
4	score_lemma	0.12	Query-document lemma overlap (Jaccard)
5	score_morph	0.08	Morphological case/gender match (Sanskrit suffix analysis)
6	score_compound	0.03	Bigram/compound match score
7	score_commentary	0.04	Commentary chunk relevance
8	score_concept	0.02	Concept overlap score
9	score_graph_centrality	0.01	Graph node degree / max_degree

Table 1: 9 Re-ranking Features and Default Weights

5.1 Adaptive Profiles

Query Type	Vector	Graph	BM25	Example
concept_short	0.45	0.20	0.05	'What is dharma?'
factual_short	0.35	0.30	0.08	'Who is Krishna?'
complex_long	0.35	0.15	0.20	Long philosophical question
concept_medium	0.40	0.22	0.08	'Explain karma yoga'
general_medium	0.40	0.18	0.12	Default

Table 2: Adaptive Weight Profiles by Query Type

6. LangGraph Orchestration

The SRAGGraphPipeline implements the full RAG pipeline as a LangGraph state machine with conditional routing for iterative query expansion.

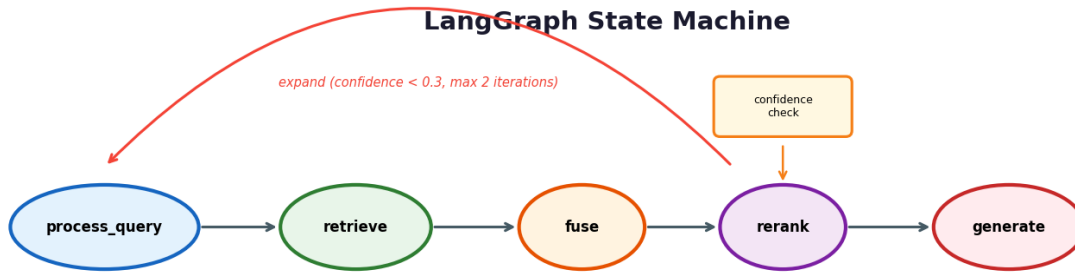


Figure 4: LangGraph State Machine

6.1 State Machine Nodes

Node	Function	Input	Output
process_query	IAST conversion + concept extraction	query	query_iast, concepts, query_type
retrieve	Vector + graph + BM25 parallel search	query_iast, concepts	vector_results, graph_results, bm25_results
fuse	Adaptive RRF fusion	all results	fused_results
rerank	9-feature linguistic re-ranking	fused_results	reranked_results, confidence
generate	MiMo v2.5 answer generation	reranked_results	answer, citations

Table 3: LangGraph State Machine Nodes

6.2 Conditional Routing

After the rerank node, a conditional edge checks the average reranking confidence. If confidence < 0.3 and iteration count < 2, the pipeline loops back to process_query with expanded concepts. Otherwise, it proceeds to generate.

6.3 Confidence Scoring

Pipeline confidence is computed as: $\text{overall} = 0.3 \times \text{retrieval} + 0.5 \times \text{reranking} + 0.2 \times \text{generation}$. Retrieval confidence uses sigmoid normalization. Generation confidence is based on citation count (1.0 if ≥ 3 citations, decreasing otherwise).

7. Answer Generation

The AnswerGenerator uses MiMo v2.5 (via OpenAI-compatible API) to generate structured markdown answers. The prompt enforces an explanation-first structure.

7.1 Prompt Design

The system prompt enforces 6 rules:

- **Explanation-first:** Model writes its own synthesis of retrieved verses, teaching the concept.
- **Verses as evidence:** Quote 1-2 lines, explain in plain language — never chain 'Verse X says...'.- **Markdown formatting:** Headings, bold key terms, bullet points.
- **Commentary as appendix:** Single most relevant commentary at the end in 'Scholarly Context'.
- **Direct answers:** Answer fully without hedging.
- **Sanskrit notation:** IAST + Devanagari in parentheses + English meaning.

7.2 Prompt Structure

Section	Content	Purpose
User Question	{query}	Original user query
Retrieved Verses	Top-5 verses with IAST + Devanagari	Primary source material
Traditional Commentary	Single highest-confidence commentary	Supplementary context

Table 4: Prompt Structure

7.3 Generation Settings

- **Model:** MiMo v2.5 (mimo-v2.5, lowercase)
- **API:** <https://api.xiaomimimo.com/v1> (OpenAI-compatible)
- **Temperature:** 0.3 (low creativity, high factual accuracy)
- **Max tokens:** 2048
- **Citation extraction:** Regex pattern `BhG\s+\d+\.\d+`

8. Evaluation Results

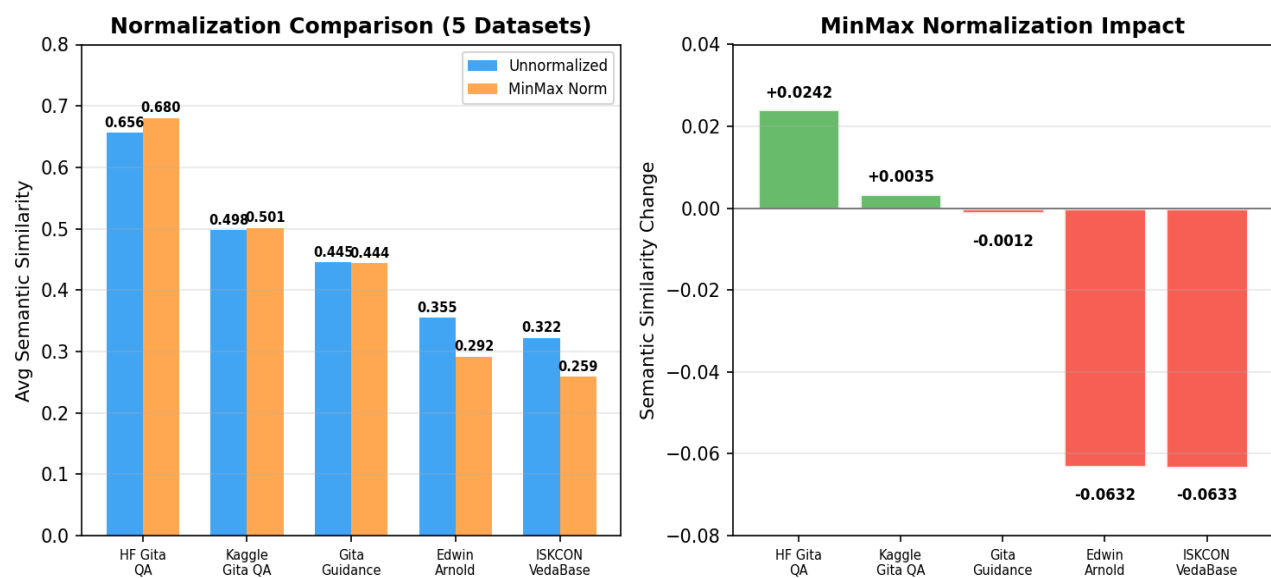


Figure 5: Evaluation Results by Dataset and Query Type

8.1 Datasets

Dataset	Type	Samples	Description
Gita Guidance QA	QA pairs	711	Modern life questions answered with Gita wisdom
Edwin Arnold QA	QA pairs	500	Factual questions from 'The Song Celestial'
ISKCON VedaBase	Verse commentary	657	Verse-by-verse Gaudiya Vaishnava commentary

Table 5: Evaluation Datasets

8.2 Results (45 samples)

Dataset	Samples	Semantic Sim.	Word Overlap
Gita Guidance QA	21	0.5013	0.2781
ISKCON VedaBase	15	0.3016	—
Edwin Arnold QA	9	0.2693	0.2769
Overall	45	0.3884	—

Table 6: Evaluation Results Summary

8.3 Analysis

- **Gita Guidance QA (0.5013):** Best match — both SRAG and the dataset answer modern-life questions with Gita wisdom. The semantic alignment is strongest here.
- **ISKCON VedaBase (0.3016):** Lower because ISKCON answers are from a single Gaudiya tradition, while SRAG uses 3 commentators (Advaita + Acintya-bhedabheda).
- **Edwin Arnold QA (0.2693):** Hardest — short factual answers (2-10 words) are difficult for semantic similarity metrics to match against longer SRAG responses.

9. Project Structure

Path	Description
configs/config.yaml	All configuration (weights, profiles, paths, API settings)
main.py	CLI entry point (preprocess, build-graph, build-indices, query)
api_server.py	FastAPI backend with /api/query and /api/health endpoints
evaluate.py	Single-dataset evaluation pipeline
evaluate_comprehensive.py	3-dataset evaluation (Gita Guidance + Edwin Arnold + ISKCON)
src/preprocessing/xml_parser.py	XML corpus parsing (3 files)
src/preprocessing/chunker.py	Chunk creation and persistence (3507 chunks)
src/preprocessing/concept_extractor.py	26 Bhagavad Gita philosophical concepts
src/preprocessing/iast_devanagari.py	IAST ↔ Devanagari conversion
src/preprocessing/morpho_extractor.py	Morphological feature extraction
src/preprocessing/graph_builder.py	Neo4j graph construction
src/retrieval/vector_store.py	FAISS vector store with bge-m3-sanskritFT
src/retrieval/bm25_retriever.py	BM25Okapi with Sanskrit suffix-aware tokenization
src/retrieval/graph_retriever.py	Neo4j graph queries (full-text + concept)
src/retrieval/hybrid_fusion.py	Adaptive RRF + weighted fusion
src/reranking/adaptive_reranker.py	5 query type profiles with dynamic weights
src/reranking/linguistic_reranker.py	9-feature re-ranking with Sanskrit morpho hints
src/reranking/feature_extractors.py	Lemma, compound, concept, centrality extractors
src/reranking/confidence.py	Pipeline confidence scoring (sigmoid + Platt)
src/generation/generator.py	MiMo v2.5 answer generation
src/generation/query_processor.py	IAST conversion + concept extraction via MiMo
src/generation/prompt_templates.py	System + user prompt templates
src/langchain_components/graph.py	LangGraph state machine (5 nodes)
src/langchain_components/state.py	TypedDict state definition
tests/	48 tests across 4 modules
web/	React + Vite frontend (saffron/gold theme)

Table 7: Project File Structure

10. Configuration

All settings are centralized in `configs/config.yaml`. Key sections:

- **data:** Paths for raw XML, processed chunks, FAISS index, graph import files
- **embedding:** Model name (bge-m3-sanskritFT), device (cpu), batch_size (8), max_length (256)
- **neo4j:** URI, credentials, database name
- **retrieval:** top_k (50), fusion_method (rrf), rrf_k (60), adaptive_weights (true)
- **reranking:** top_n (5), method (weighted), adaptive (true), 9 feature weights
- **adaptive_profiles:** 5 query type profiles with retrieval and reranking weights
- **generation:** provider (mimo), model (mimo-v2.5), temperature (0.3), max_tokens (2048)
- **langgraph:** enabled (true), max_iterations (2), confidence_threshold (0.3)
- **confidence:** pipeline_weights (retrieval: 0.3, reranking: 0.5, generation: 0.2)

11. API Reference

11.1 POST /api/query

Field	Type	Description
query	string	User query in any language

Table 8: Request Schema

Field	Type	Description
query	string	Original query
query_iast	string	IAST transliteration
query_devanagari	string	Devanagari rendering
concepts	list[str]	Extracted concepts
answer	string	Generated markdown answer
verses_cited	list[str]	Verse references cited
top_verses	list[dict]	Top 5 retrieved verses
pipeline_confidence	dict	Confidence scores
query_type	string	Detected query type

Table 9: Response Schema

11.2 GET /api/health

Returns: { status: 'ok', chunks: int, pipeline_type: 'langgraph' | 'standard' }

12. Getting Started

12.1 Prerequisites

- Python 3.11+
- Neo4j 5.x running on bolt://localhost:7687
- MiMo API key (MIMO_API_KEY in .env)
- ~4 GB RAM for embedding model

12.2 Installation

```
pip install -r requirements.txt
echo "MIMO_API_KEY=your_key" > .env
```

12.3 Build Indices

```
python main.py preprocess
python main.py build-graph
python main.py build-indices
```

12.4 Query

```
python main.py query --query "What is dharma?" --langgraph
```

12.5 Run Tests

```
pytest tests/ -v
```

Expected: 48 passed

12.6 Web UI

```
python api_server.py
```

Open <http://localhost:8000>